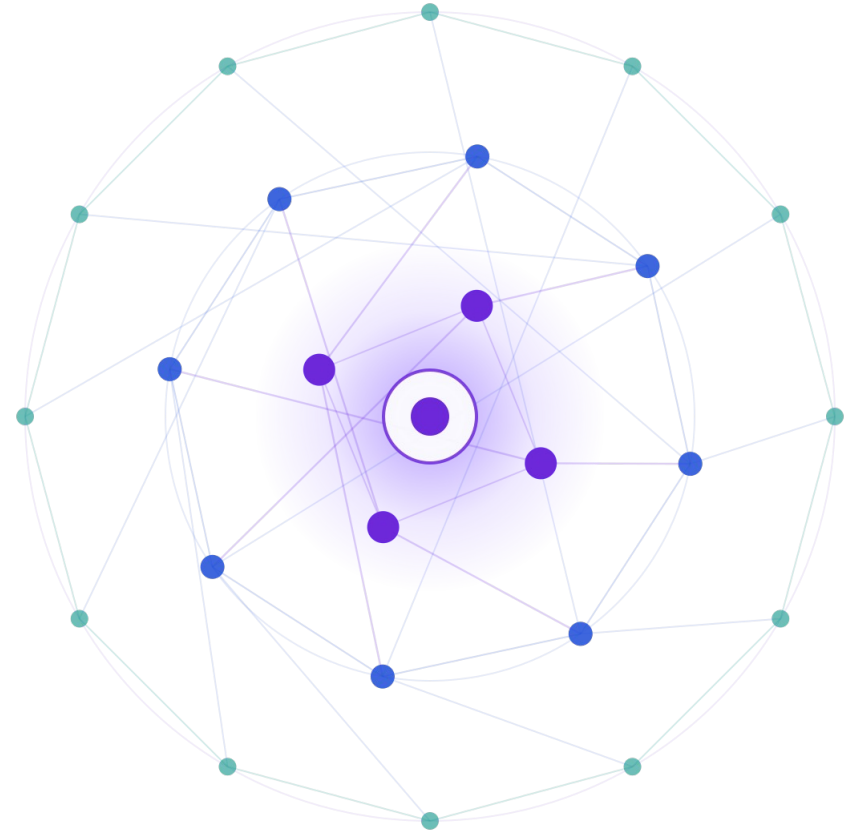


Generative AI Hackathon

3rd Edition 2026 – 27

- Theme ID - Theme 2: Smart Guided Troubleshooting Engine
- Team Name - <your team name>
- College Name - <your college name>
- Member Name & Email 1-
- Member Name & Email 2-
- Member Name & Email 3-
- Member Name & Email 4-
- Submission GitHub link -



Theme

A user describes a device problem in plain language — the app identifies the exact issue and returns a short set of trusted troubleshooting steps, plus a button that opens the relevant setting.

1 Understand

An LLM turns an informal complaint into a structured issue: domain = battery, issue = fast_drain.

2 Retrieve

The backend looks that issue up in a small, curated troubleshooting table — no invented fixes.

3 Act

The UI shows 3–5 clear steps and a verified action button where one exists.

Typical input

“My phone battery is draining very fast.”



What comes back

Battery → Fast Drain · 5 steps · “Open Battery Settings”

Existing Solutions & Gaps

How people troubleshoot today

Web search & forums

Dozens of conflicting threads, mostly written for a different device or OS version.

Manufacturer support pages

Accurate, but organised by product manual structure — not by how users describe a problem.

General-purpose chatbots

Fluent answers, but nothing anchors them to verified steps for that specific issue.

Where it falls short

Language mismatch

Users say “phone is dying fast”; documentation says “battery discharge optimisation”.

Unverified instructions

Free-form models invent settings paths and deep links that do not exist on the device.

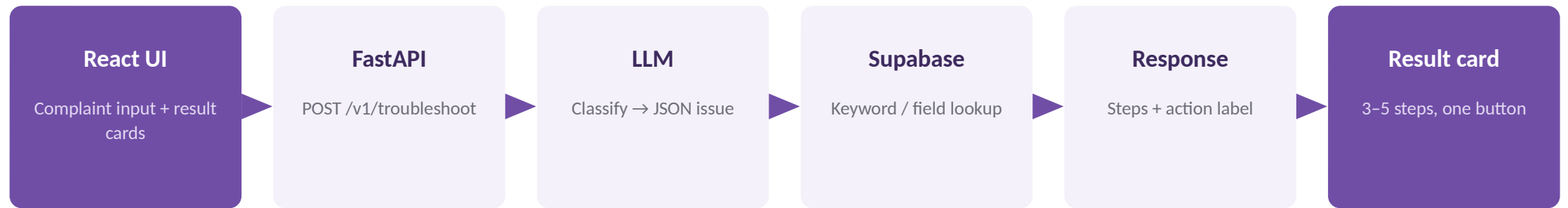
Answer, but no action

Even a correct answer leaves the user hunting through five levels of settings menus.

The gap we close: informal language → a known issue → curated steps → a working action button.

Our Solutions & Architecture Diagram

React → FastAPI → LLM → Supabase → FastAPI → React. One request path, easy to demo, debug and explain.



Structured output, not prose

The LLM returns only {domain, issue, confidence}. Nothing it writes reaches the user as advice.

Knowledge-grounded answers

Every step shown comes from our troubleshooting table, so the content is reviewable and fixable.

Honest no-match path

If confidence is low or no record matches, we say so instead of guessing.

Demo & Product Walkthrough

One complaint, followed end to end — then two more scenarios including a deliberate no-match.

1. What the user types

“my battery is dying very fast”

2. What the LLM returns

```
{ "domain": "battery",  
  "issue": "fast_drain",  
  "confidence": 0.92 }
```

3. What the user sees

Battery • Fast Drain

- 1 Check Battery Usage for the top-draining app
- 2 Turn on Adaptive / Power Saving mode
- 3 Lower screen brightness and screen timeout
- 4 Restrict background activity for heavy apps

Open Battery Settings

Scenario A • Battery

Fast drain — full match, action button available.

Scenario B • Performance

Phone feels slow — match with steps, no deep link shown.

Scenario C • No match

Out-of-scope complaint — clear “not supported yet” response.

Tools and tech stack used

Chosen for speed of build and ease of explanation — every piece is something the team can already debug.

FRONTEND

React + Vite + Tailwind

Fast UI iteration and familiar web development.

BACKEND

Python + FastAPI

A thin API layer that is easy to wire to an LLM.

DATABASE

Supabase (PostgreSQL)

One troubleshooting table; the team already knows it.

AI

Gemini / existing LLM API

Turns the user's complaint into structured data.

VALIDATION

Pydantic

Keeps LLM output predictable and rejects bad shapes.

DEVELOPMENT

Antigravity

Generates boilerplate, debugs, explains unfamiliar code.

Deliberately out of v1:

embeddings, pgvector, a separate vector database, Redis, microservices, agent frameworks, Kubernetes, custom auth.

Impact & Use case

A narrow product that works reliably beats a broad one that half-works.

20–30

curated troubleshooting records at launch,
each one reviewed by a human

4

device domains covered first: Battery, Display,
Camera, Performance

1 tap

from the answer to the correct settings screen,
where a verified link exists

Who it helps

Everyday users

Get a specific fix without reading
five forum threads first.

Non-technical users

Plain-language input and a button
instead of a settings maze.

Support teams

Deflect repeat first-line questions
to a consistent, reviewed answer.

Retail & service desks

Staff give the same trusted steps
every time, with no training
burden.

Innovation highlights, results and limitations

What is genuinely new, what already runs, and what we know it cannot do yet.

Innovation highlights

- LLM used as a classifier, not an answer generator — it names the issue, the database supplies the fix.
- Action layer: answers end in a verified deep link or button, not just text.
- A no-match response is treated as a real feature, not an error state.

Results so far

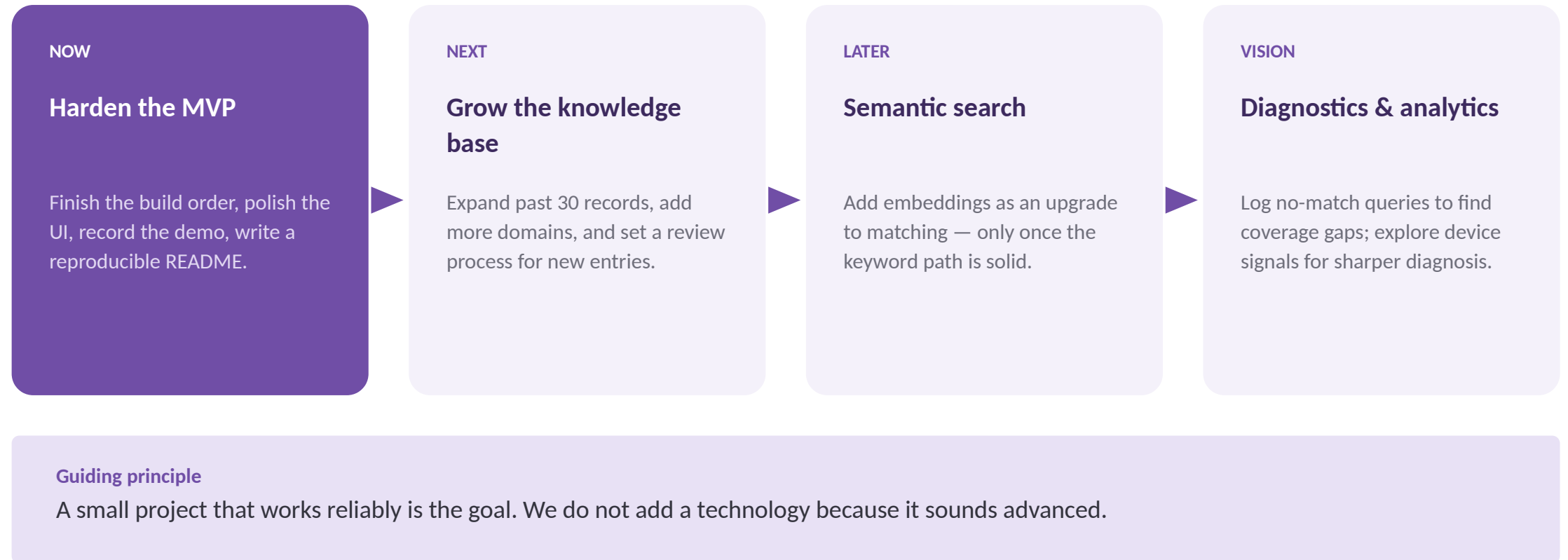
- Working vertical slice: input → API → LLM → Supabase → result card.
- Structured JSON classification with a confidence score on every request.
- Starter knowledge base across four device domains, hand-written and reviewed.

Known limitations

- Coverage is limited to the issues in the table; anything else returns no-match.
- Keyword and field matching misses unusual phrasings that semantic search would catch.
- Deep links are device- and OS-dependent, so they are only shown where verified.

What's next

Finish the core demo first; add advanced retrieval only when there is a clear reason for it.



Brownie points slide(differentiation)

Four choices that separate this from a wrapper around a chat model.

1 Grounded by construction

The model never writes the fix. It only selects which known issue the user is describing, so every instruction shown has been reviewed by a person.

2 No invented deep links

Action buttons appear only for links we have verified on a real device. Where we have none, we show steps and say so.

3 Designed to say “I don't know”

Low confidence or no matching record produces an explicit no-match response instead of a plausible-sounding guess.

4 Explainable in one diagram

A second-year team can walk a judge through every hop in the request path — which also makes it debuggable under demo pressure.

Checklist- Updated on Public GitHub

Update each line before submission.

☐

Working prototype code

Public or shared GitHub repository containing the React and FastAPI code.

Y / N

☐

README with setup instructions

Reproducible steps: environment variables, Supabase schema, how to run both services.

Y / N

☐

Demo video, max 5 minutes

YouTube or Drive link showing all three demo scenarios end to end.

Y / N

☐

Presentation file

This deck, exported as PPT or PDF and uploaded with the submission.

Y / N

Thank you

Organised by the Language AI Team and the PRISM Team, Samsung R&D Institute India